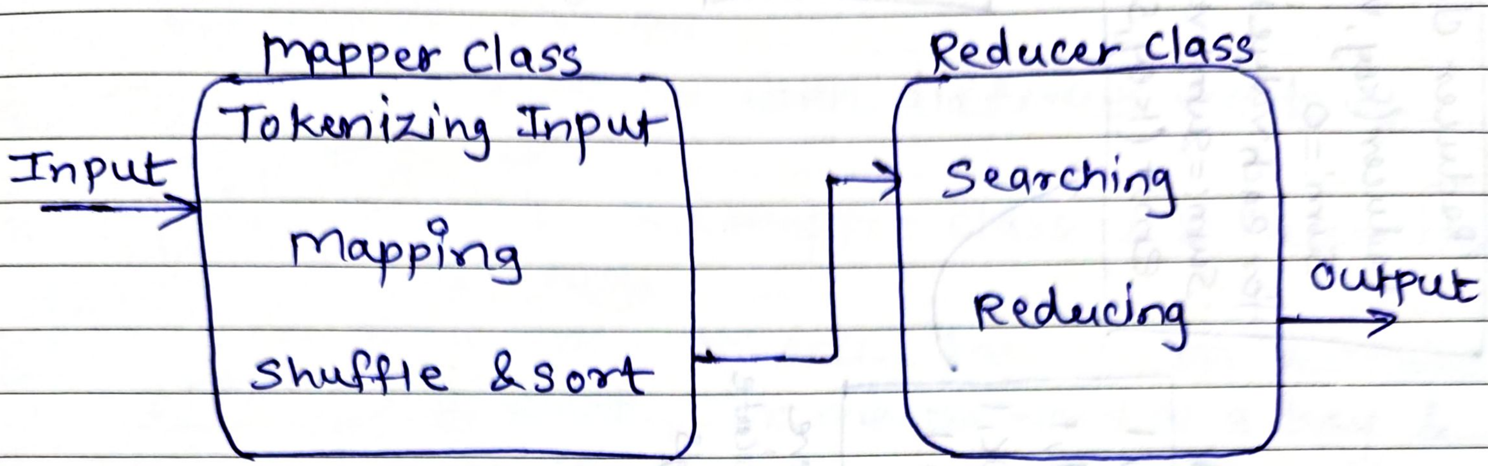
Date :

Basic of MapReduce Algorithm



Different Tasks in MapReduce Algorithms.

The mapreduce programs have two tasks

- Map task
- Reduce task

Map task is done by the mapper class & Reduce task is done by reducer class.

Mapper class takes the input, tokenizes it, maps & sorts it. The output of mapper class is used as input by reducer class, which in turn searches matching pairs and reduces them.

Date :

1. a duck is a bird

mapper class

Takeline from input & tokenizes it

for each word in line
emit(word,1)

a, 1
duck, 1
is, 1
a, 1
bird, 1

mapper output

a, 1
a, 1
bird, 1
duck, 1
is, 1

Sorted output

a,1,1
bird,1
duck,1
is,1

Remove Duplicate Keys

Reducer class

Reducer(key, value)
Sum:=0
for each value in key:
Sum:=sum+value
emit (key,Sum)

a,2
bird,1
duck,1
is,1

final output

Date :

MapReduce Task Example

Word Count Problem

It takes the string with different words of each type.

Let us see how the mapper class & reducer class work for this task

→ The mapper class tokenize the strings & make a Sorted list of words. It marks word as a key & number as value.

→ The reducer class takes the list and count the no. of entries of each word; finally creates another list with keys & their count values.

Date :

Matrix Vector Multiplication by Map Reduce.

$$A = \begin{array}{c|c} & j \\ & \begin{array}{cc} 0 & 1 \end{array} \\ \hline i \; \begin{array}{c} 0 \\ 1 \end{array} & \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \end{array} \quad 2 \times \textcircled{2}
\qquad
B = \begin{array}{c|c} & k \\ & \begin{array}{cc} 0 & 1 \end{array} \\ \hline j \; \begin{array}{c} 0 \\ 1 \end{array} & \begin{bmatrix} 2 & 1 \\ 5 & 1 \end{bmatrix} \end{array} \quad \textcircled{2} \times 2$$

i K   i; j

Whenever we are multiplying two matrices There is a rule the total no. of columns in the first matrix should be equal to the total no. of rows from the second matrix. And if they are equal then only we can multiply two matrices.

If we multiply this two matrices it looks like

$$\begin{bmatrix} M_{00} & M_{01} \\ M_{10} & M_{11} \end{bmatrix}$$

Steps for multiplying two matrices using map reduce concept.

① Map task:
Convert data in key value pairs.
According to algorithm.

for matrix 1 $\longrightarrow$ A

$$(\underbrace{(i, k)}_{\text{key}}, \underbrace{(A, j, A_{ij})}_{\text{value}})$$

Date :

for matrix 2 → B

$$\Big(\underbrace{(i,k)}_{\text{key}},\ \underbrace{(B,j,B_{jk})}_{\text{value}}\Big)$$

for matrix A

$A_{00} = 1$ $\quad$ $((0,0),(A,0,1))$

A_{ij} $\quad$ $((0,1),(A,0,1))$

$A_{01} = 2$ $\quad$ $((0,0),(A,1,2))$

A_{ij} $\quad$ $((0,1),(A,1,2))$

$A_{10} = 3$ $\quad$ $((1,0),(A,0,3))$

A_{ij} $\quad$ $((1,1),(A,0,3))$

$A_{11} = 4$ $\quad$ $((1,0),(A,1,4))$

$A_{ij} =$ $\quad$ $((1,1),(A,1,4))$

Date :

Matrix B

$B_{00} = 2$ $\quad ((0,0), (B,0,2))$

B_{jk} $\quad ((1,0), (B,0,2))$

$B_{01} = 1$ $\quad ((0,1), (B,0,1))$

B_{jk} $\quad ((1,1), (B,0,1))$

$B_{10} = 5$ $\quad ((0,0), (B,1,5))$

B_{jk} $\quad ((1,0), (B,1,5))$

$B_{11} = 1$ $\quad ((0,1), (B,1,1))$

B_{jk} $\quad ((1,1), (B,1,1))$

Date :

(2) Combiner Task

Used to combine key value pairs to reduce network Congestion.

(0,0), (A,0,1), (A,1,2), (B,0,2), (B,1,5)
(0,1), (A,0,1), (A,1,2), (B,0,1), (B,1,1)
(1,0), (A,0,3), (A,1,4), (B,0,2), (B,1,5)
(1,1), (A,0,3), (A,1,4), (B,0,1), (B,1,1);

(3) Reduce Task

for (0,0)

→ (A,0,1) (A,1,2)
(B,0,2) (B,1,5)

1×2 2×5

2 + 10 = 12

Element located at f00 = 12

for (0,1) →
(A,0,1) (A,1,2)

(B,0,1) (B,1,1)

1×1 2×1

1 + 2 = 3

Date :

for (1, 0) $\rightarrow$ (A, 0, $\boxed{3}$) (A, 1, $\boxed{4}$)

f_{10} (B, 0, $\boxed{2}$) (B, 1, $\boxed{5}$)

3×2 4×5

6 + 20 = 26

for (1, 1) $\rightarrow$ (A, 0, $\boxed{3}$) (A, 1, $\boxed{4}$)

f(1, 1) (B, 0, $\boxed{1}$) (B, 1, $\boxed{1}$)

3×1 4×1

3 + 4 = 7

final matrix <u>f</u> $\rightarrow$ $\begin{bmatrix} f00 & f01 \\ f10 & f11 \end{bmatrix}$

$\begin{bmatrix} 12 & 3 \\ 26 & 7 \end{bmatrix}$

1 2 2 1

3 4 5 1

2+10 1+2

$\begin{bmatrix} 12 & 3 \\ 26 & 7 \end{bmatrix}$

Date :

Relational algebra operations using mapreduce.

- Selection.
- Projection
- Union
- Intersection
- Difference
- Natural Join
- Grouping & Aggregation

(1) Selection Algorithm

```
Map (key, value):
    for tuple in value:
        if tuple satisfies condition:
            emit (tuple, tuple)

reduce (key, values):
    emit (key, key)
```

MAP worker 1

A	B
1	2
3	1

A	B
1	2
3	5

MAP worker 2

A	B
2	3
3	5

A	B
1	1
2	1

Big data divides entire data in various parts each part will be process separately & parallely.

Date :

☆ Selection Condition B<=2

MW1

key	value
(1,2)	(1,2), (1,2)
(3,1)	(3,1)
(1,2)	(1,2)

add

MW2

key	value
(1,1)	(1,1)
(2,1)	(2,1)

Map worker also checks duplicacy of data

☆ A hash function will be applied :

MW1

key	value
(1,2)	(1,2),(1,2)

key	value
(3,1)	(3,1)

MW2

key	value
(1,1)	(1,1)

key	value
(2,1)	(2,1)

Date :

To reduce task (swapping will be done)

To discard the redundancy that can cause by duplicate tuples

RW1

key	value	key	value
(1,2)	(1,2) (1,2)	(1,1)	(1,1)

RW2

key	value	key	value
(3,1)	(3,1)	(2,1)	(2,1)

★ To join two tables

RW1

key	value
(1,2)	(1,2) (1,2)
(1,1)	(1,1)

RW2

key	value
(3,1)	(3,1)
(2,1)	(2,1)

★

RW1

A	B
1	2
1	1

RW2

A	B
3	1
2	1

Date :

Projection

If we have 3 attributes name, age, & salary & ~~Let's say~~ we want to Project only name & age So in that case we can use projection.

Let's say S be subset Containing selected attributes.

```
map (key, value):
    for tuple in value:
        ts = tuple with only attributes in S
        emit (ts, ts)

reduce (key, values):
        emit (key, key)
```

Project: (B, C).

map worker 1

A	B	C
1	2	3
3	1	4

A	B	C
2	2	3
4	3	4

MAP worker 2

A	B	C
5	7	2
7	1	1

A	B	C
3	2	5
4	1	0

S will contain all records which are under B & C

Date :

MW1

key	value
(2,3)	(2,3), (2,3)
(1,4)	(1,4)
(3,4)	(3,4)

MW2

key	value
(7,2)	(7,2)
(1,1)	(1,1)
(2,5)	(2,5)
(1,0)	(1,0)

(key, value)

Hash function will divide entire table in parts

MW1

key	value
(2,3)	(2,3)(2,3)
(1,4)	(1,4)

key	value
(3,4)	(3,4)

MW2

key	value
(7,2)	(7,2)
(1,1)	(1,1)

key	value
(2,5)	(2,5)
(1,0)	(1,0)

Swapping is done because to remove redundancy present in both map workers.
This can be done with help of reducers.

RW1

key	value
(2,3)	(2,3)(2,3)
(1,4)	(1,4)

key	value
(7,2)	(7,2)
(1,1)	(1,1)

RW2

key	value
(3,4)	(3,4)

key	value
(2,5)	(2,5)
(1,0)	(1,0)

'Swap the table with adjacent workers

Date :

RW1

key	value
(2,3)	(2,3) (2,3)
(1,4)	(1,4)
(7,2)	(7,2)
(1,1)	(1,1)

RW2

key	value
(3,4)	(3,4)
(2,5)	(2,5)
(1,0)	(1,0)

RW1

B	C
2	3
1	4
7	2
1	1

RW2

B	C
3	4
2	5
1	0

Union

Union of two tables will be collection of all distinct values which are present in both the tables.

Inside map task we will be iterating over the values in the relation & store each value in the temporary variable tuple. & simply emit key value in the form of (tuple, tuple)

reduce section we will ~~reducing~~ key part that we emitting collected from map task

Date :

Union Algorithm

union of 2 tables will be collection of all distinct values which are present in tuple.

map(key, value):
    for tuple in value:
        emit (tuple, tuple)

reduce(key, value)
    emit (key, key)

map worker 1

Table 1

A	B
1	2
3	1

Table 2

A	B
1	2
2	1

map worker 2

Table 1

A	B
2	3
4	5

Table 2

A	B
1	1
2	1

step 1

mw1

key	value
(1,2)	(1,2), (1,2)
(3,1)	(3,1)
(2,1)	(2,1)

mw2

key	value
(2,3)	(2,3)
(4,5)	(4,5)
(1,1)	(1,1)
(2,1)	(2,1

Why we divide the table because previous key, value is divided in 2 parts

Date :

Step2: Apply Hash function which will divide the entire key value ~~part~~ pair is done from the map workers by 2. This ~~will remove data duplicacy~~ discard redundancy that may occur due to duplicacy.

MW1

key	value
(1,2)	(1,2)(1,2)
(3,1)	(3,1)

key	value
(2,1)	(2,1)

MW2

key	value
(2,3)	(2,3)
(4,5)	(4,5)

key	value
(1,1)	(1,1)
(2,1)	(2,1)

Step3: Convert map workers to reduce workers. Switch the table ~~m~~ among different map workers. Reducer worker will ~~swap~~ switch one of the table from 1st map worker with the one of the table from 2nd mapworker This way discard redundancy.

RW1

key	value
(1,2)	(1,2)(1,2)
(3,1)	(3,1

key	value
(2,3)	(2,3)
(4,5)	(4,5)

RW2

key	value
(2,1)	(2,1)

key	value
(1,1)	(1,1)
(2,1)	(2,1)

Date :

RW1

key	value
(1,2)	(1,2)(1,2)
(3,1)	(3,1)
(2,3)	(2,3)
(4,5)	(4,5)

RW2

Key	value
(2,1)	(2,1),(2,1)
(1,1)	(1,1)

reducer worker will emit key part

RW1

A	B
1	2
3	1
2	3
4	5

RW2

A	B
2	1
1	1

In map task we will iterating through values & will store each value in temporary variable tuple & will simply emit(key, value) in form of (tuple, tuple)
Inside reduce task it will check the values ~~pair~~ part in key value pair contain more than one tuple which means length of the value part is more then one tuple which means the length of value part is more then one then in that case it clears out that the key has got repeated & hence we can conclude it as intersection value.

Date :

some of the tuples same in both table The result of intersection will be (same tuples between two)

Intersection Algorithm

```
map (key, value):
    for tuple in value:
        emit(tuple, tuple)

reduce (key, values):
    if values == [key, key]
```

map worker 1

Table1

A	B
1	2
3	1

Table2

A	B
1	2
2	1

map worker 2

Table1

A	B
2	3
4	5

Table2

A	B
1	1
2	1

1 | 2 record is common in both table

Step 1 — Convert record in key, value Pair.

MW1

key	value
(1,2)	(1,2) (1,2)
(3,1)	(3,1)
(2,1)	(2,1)

MW2

key	value
(2,3)	(2,3)
(4,5)	(4,5)
(1,1)	(1,1)
(2,1	(2,1)

Step 2:

Date :

Hash function will be applied ~~for~~ key value table for every mapworker will be divided by two.

mw1

key	value
(1,2)	(1,2)
	(1,2)
(3,1)	(3,1)

key	value
(2,1)	(2,1)

mw2

key	value
(2,3)	(2,3)
(4,5)	(4,5)

key	value
(1,1)	(1,1)
(2,1)	(2,1)

Swap one of the key value table from map worker1

Step 3:

RW1

key	value
(1,2)	(1,2)(1,2)
(3,1)	(3,1)

key	value
(2,3)	(2,3)
(4,5)	(4,5)

RW2

key	value
(2,1)	(2,1)

~~Start~~

key	value
(1,1)	(1,1)
(2,1)	(2,1)

Step 4
Single key value table in both reducer, worker

RW1

key	value	
(1,2)	(1,2)(1,2)	
(3,1)	(3,1)	x
(2,3)	(2,3)	x
(4,5)	(4,5)	x

RW2

key	value	
(2,1)	(2,1)	x
(1,1)	(1,1)	

Date :

If length of "values" is greater than 1, then emit the key part it

RW1

key	value
(1,2)	(1,2)(1,2)

RW2

key	value
(2,1)	(2,1)(2,1)

RW1

A	B
1	2

RW2

A	B
2	1

All the value available in T1 except in T2

Date:

Difference Algorithm

```
map (key, valle)                      —— Current relation R in table T1
  if (key == R)
     for tuple in value:
        emit (tuple, R)
  else
     for tuple in valee:              Will iterate over the valles in relation S.
        emit (tuple, S)

reduce (key, values):
    if values == [R]:
        emit (key, key)
```

Map Worker 1

Table 1

A	B
1	~~2~~ ✗
3	1 ✓

Table 2

A	B
1	2
2	1

map worker 2

Table 1

A	B
2	3
4	5

A	B
1	1
2	1

According to difference operation the value in first & second table

A	B
1	2

A	B
1	2

Relation R — Table T_1

S — Table T_2

- Both T_1 T_2
- only T_1
- only T_2

Date :

Step 1

MW1

~~(1,2)~~

key	value
(1,2)	[T_1,T_2]
(3,1)	[T_1]
(2,1)	[T_2]

MW 2

key	value
(2,3)	[T_1]
(4,5)	[T_1]
(1,1)	[T_2]
(2,1)	[T_2]

Step2 – Apply Hash function

Each mapworker will be divided into two table.

MW1

key	value
(1,2)	[T_1,T_2]
(3,1)	[T_1]

key	value
[2,1]	[T_2]

MW2

key	value
(2,3)	[T_1]
(4,5)	[T_1]

key	value
(1,1)	[T_2]
(2,1)	[T_2]

Swapping

Step3 – ~~maps~~ Swapping will be done ~~with~~ by reducer worker

RW1

key	value
(1,2)	[T_1,T_2]
(3,1)	[T_1]

key	value
(2,3)	[T_1]
(4,5)	[T_1]

RW 2

key	value
(2,1)	[T_2]

key	value
(1,1)	[T_2]
(2,1)	[T_2]

step 4 —

Date :

find out duplicate keys.

RW1

key	value
(1,2)	[T1, T2]
(3,1)	[T1]
(2,3)	[T1]
(4,5)	[T1]

RW2

key	value
(2,1)	[T2]
(1,1)	[T2]
(2,1)	[T2]

Eliminate the keys that are present both in T1 & T2 as well as only from T2

MW1

key	value
(3,1)	[T1]
(2,3)	[T1]
(4,5	[T1]

MW2

RW1

A	B
3	1
2	3
4	5

RW2

will be empty.

Date :

Hadoop limitations

① Security Problem

Hadoop does not implement encryption-decryption at the storage as well as n/w levels

Solution
Spark implements AES-based encryption for RPC connections. we should enable RPC authentication to enable encryption. It should be properly configured.

② Issue with small files
Small files are the major problem in HDFS
A small file is smaller than HDFS block size (128 MB)
If we are storing huge no small files.
HDFS is for working properly with small no of large file for large data sets rather than large no. of small files.
If there are too many small files then Name Node will get overload since it stores the namespace of HDFS.

Date :

Solution to this, merge the small files to create bigger files & then copy bigger files to HDFS.

Hadoop Archives file works on the top of HDFS.

(3) Slow Processing Speed

In hadoop with a parallel & distributed algorithm map reduce process large data sets. It requires lot of time to perform these tasks. Data is distributed & processed over the cluster.

Solution to this, Hadoop Spark. It's memory processing is faster. Spark 100 times faster than mapreduce as it processes everything in memory.

(4) Support for batch processing only.

It does not process streamed data, hence overall performance is slower

To solve this we hand spark stream processing Flink improves the overall performance as it provides single run time for streaming as well as batch processing.

Flink uses native closed loop iteration operators which make use machine learning & graph processing faster.

Hive used for querying & analyzing large dataset stored in hadoop files.

Date :

Apache spark Support Stream Processing
Apache flink provides single run time for the streaming as well as batch Processing

(5) Not easy to use

In hadoop map reduce developers need to hand code for each and every operation which make it difficult ~~for~~ to work.
Map reduce has no interactive mode but adding one such as Hive & Pig makes working with map/reduce a little easier for adopters.

Hive QL translate SQL queries into mapreduce job

Solution
We can use spark, it has interactive mode so that developers & users can have intermediate feedback for queries & other activities

Apache Spark.
Open Source tool
developed in 2012 at AMPLab at UC Berkeley
It works in memory
It is designed to use RAM for ~~th~~ caching & processing the data

Date :

Spark perform different types of big data workloads like

Batch processing
Real time stream processing
machine learning
Graph computation
interactive queries.